Sofia: A Quantum–Classical Deep RL for Kubernetes Autoscaling

Constantinos Bitsakos
CSLAB, NTUA, Greece
kbitsak@cslab.ece.ntua.gr

Dimitrios Tsoumakos
DBLAB, NTUA, Greece
dtsouma@mail.ntua.gr

Ioannis Konstantinou
Dept. of Informatics and Telecommunications
University of Thessaly, Greece
ikons@uth.gr

Nectarios Koziris
CSLAB, NTUA, Greece
nkoziris@cslab.ece.ntua.gr

Theodoros Andronikos
Ionian University, Greece
andronikos@ionio.gr

***Abstract*—Autoscaling a Redis cluster on Kubernetes is a control problem with delays. A new pod helps only after it has started and joined the cluster. A threshold rule that reacts to the current CPU level, such as the Horizontal Pod Autoscaler (HPA) scaling on CPU, therefore tends to add capacity after the load has already risen. Deep Reinforcement Learning (Deep RL) agents such as DInos learn proactive policies from a window of recent metrics and adapt them to a new environment by fine-tuning. In the DInos implementation, almost all of the Q-network's parameters sit in a recurrent encoder, and fine-tuning updates all of them. In this paper, we introduce Sofia, a hybrid quantum–classical Deep RL agent in which a variational quantum circuit replaces part of the DInos Q-network. To evaluate it, we build a simulator in which latency follows from a queueing model of the Redis pods, and pod start-up and data rebalancing take time. Before any agent is compared, the simulator must pass sanity gates: trivial fixed policies have to lose to a tuned threshold autoscaler. Our protocol compares Sofia with DInos retrained in the same simulator, a threshold policy modeled on the HPA, and a classical network with as many trainable parameters as Sofia. It measures SLA violations, tail latency, sample efficiency, adaptation and model size, and checks whether the ranking holds on a real cluster. All circuits run on a classical simulator, and we make no claim about quantum speed-up.**

***Index Terms*—Kubernetes, autoscaling, deep reinforcement learning, quantum machine learning**

I. Introduction

An in-memory key–value store such as Redis is often the first component to feel a change in load. When it runs on Kubernetes, the operator chooses how many pods serve the cluster. Too few pods and requests queue up, so tail latency grows and the service misses its latency objective [1]. Too many pods and the operator pays for capacity that sits idle. The default tool is the Horizontal Pod Autoscaler (HPA). It runs a control loop, by default every 15 seconds, and sets the replica count from the ratio between the current value of a metric and its target [2]. When the metric is CPU utilization, as in the threshold baseline we use, the rule responds to load that has already arrived. A pod added now serves traffic only after it is scheduled, its image is pulled and it becomes ready [3], [4], and in Redis Cluster hash slots must then be moved to it before it takes its share of the keys [5], [6].

Learning-based autoscalers aim to be proactive: they try to act before the load arrives. DERP [7] applied Deep Reinforcement Learning (Deep RL) to elastic resource provisioning. DInos [8] targets autoscaling that generalizes across stateless cloud applications. As published, its agent observes load, CPU, memory, latency and the number of pods over a window of recent steps, processes the window with a Long Short-Term Memory (LSTM) network [9], and chooses every 15 seconds whether to add a pod, remove one or do nothing. The paper pre-trains the agent in a simulator and reports fine-tuning it for new environments, including a Kubernetes cluster running Redis. The Q-network has **[TBD]** trainable parameters, almost all of them in the recurrent encoder, and fine-tuning updates the entire network.

Variational quantum circuits (VQCs) have been used as function approximators in reinforcement learning, both as policies [10] and as Q-functions [11], and hybrid networks that put a trainable circuit after a pre-trained classical network have been used for transfer learning [12]. The circuits in these studies are small, but a small parameter count is weak evidence on its own. Skolik et al. report that architectural and hyperparameter choices matter more than the number of parameters [11], so a small classical network could match a small circuit. We therefore treat parameter efficiency as a hypothesis and test it against a classical control with the same number of trainable parameters. We also keep timing claims apart from it. Our circuits run on a classical simulator, and simulator wall-clock time says nothing about speed on quantum hardware.

The comparison also depends on the environment, so we build a simulator for this study. Arrivals enter a queue served by the ready pods, so the number of pods affects capacity, latency and CPU. New pods start after a delay, and capacity drops for a while after each change while the cluster rebalances. The reward is based on the service-level objective (SLO) and the cost of the pods. Before any agent is compared, the simulator has to pass fixed sanity gates: always using the minimum or the maximum number of pods, or the best static pod count, must lose to a tuned threshold policy, and an oracle that knows future arrivals gives an upper bound on the reward.

The simulator's parameters are provisional until a calibration pilot on a real cluster.

In this paper, we introduce Sofia, a hybrid quantum–classical Deep RL agent for Kubernetes autoscaling. Sofia keeps the three DInos actions and replaces part of the Q-network with a VQC of **[TBD]** qubits. Its input differs from the published DInos state. We use a common interface: every learning agent in this paper observes the same window of six signals that can be measured on a real cluster: completed operations per second, CPU utilization, p95 latency, failed requests per second, and the numbers of ready and pending pods. Memory is left out because, in our simulator, it would only mirror the request backlog.

Building on these observations, we make four contributions. First, we describe a simulator for Redis on Kubernetes in which the reward depends on the scaling action through queueing, start-up delay and rebalancing, together with the sanity gates and the oracle bound that every agent comparison has to pass (Section V). Second, we design Sofia and specify its circuit, its encoding of the metric window and its training procedure (Section VII). Third, we define an evaluation protocol and apply it (Section VIII). We compare Sofia with DInos retrained in the same simulator, a threshold policy modeled on the HPA, trivial fixed policies, the oracle bound and a parameter-matched classical control. Where only a small part of the network is adapted, we add a classical control that fine-tunes the same number of parameters on a frozen encoder. Every learning agent gets the same hyperparameter search budget. The workloads range from a periodic load with noise and spikes, as in DInos, to a bursty Poisson-based load, a replayed production trace and a load whose pattern shifts mid-run. We run **[TBD]** seeds and report bootstrap 95% confidence intervals for SLA violations, p95 latency, steps to reach a target reward, adaptation after a shift, trainable parameters, checkpoint size, peak training memory and training time. Fourth, we test whether the ranking of the agents obtained in simulation holds on a Kubernetes cluster running Redis. Our circuits run on a classical simulator, and none of our results is a claim about quantum speed-up.

The rest of the paper is structured as follows. Section II gives background on Kubernetes autoscaling, Redis and variational circuits. Section III reviews related work. Section IV describes the system and Section V the simulator. Sections VI and VII present the agents, the baselines and the quantum component. Section VIII reports the evaluation, Section IX discusses limitations and Section X concludes.

II. Background

III. Related Work

a) Autoscaling of cloud applications.: Lorido-Botran et al. [13] survey autoscaling techniques for elastic applications in the cloud. In Kubernetes the standard mechanism is the HPA. It compares a metric such as CPU utilization with a target, scales the replica count by their ratio, ignores changes within a tolerance, and holds back scale-down decisions for a stabilization window [2]. During an initialization period after a pod starts, its CPU samples count only if the pod is ready and the sample was taken while it was ready [2]. AHPA, an autoscaler used in Alibaba Cloud's Kubernetes service, models the response time of pods with M/M/1 and M/M/c queues and finds the M/M/c model more accurate when there are few pods [14]. The AHPA description is a preprint. We use a threshold policy in the spirit of the HPA as a baseline and a per-pod queueing model inside our simulator.

b) Deep reinforcement learning for autoscaling.: DERP [7] applied Deep RL to elastic resource provisioning in the cloud. DInos [8] compares against an enhanced version of DERP. As published, the DInos agent observes a window of recent metrics (load, CPU, memory, latency and the number of pods), processes it with an LSTM [9], and chooses every 15 seconds whether to add a pod, remove one or do nothing. The paper pre-trains the agent in a simulator and reports fine-tuning it in the target environment, including a Kubernetes cluster serving Redis. Taylor and Stone [15] survey transfer in reinforcement learning and frame the question DInos asks: whether starting from a trained policy beats training from scratch in the new task. Double Q-learning [16] reduces the overestimation of action values in deep Q-networks. Sofia and the retrained DInos baseline use its target. Sofia keeps the DInos actions. For the state, we do not reuse the published DInos features. Sofia and the retrained DInos baseline observe the same six signals that can be measured on a real cluster (Section V), so the two agents differ only in the function approximator.

c) Performance and workload models for simulation.: Queueing theory gives the standard link between load, capacity and response time [17], and Little's law [18] relates the number of requests in a system to their arrival rate and their time in the system. For latency-critical services the tail matters more than the mean [1]. Measured tails of servers such as Memcached are worse than queueing models predict, because of interference from the hardware, the operating system and the application [19], and queueing and scheduling delays cause violations of sub-millisecond targets at high utilization [20]. On the workload side, Atikoglu et al. [21] analyze traces of a large Memcached deployment, Bodik et al. [22] characterize and generate workload spikes for stateful services, and Markov-modulated Poisson processes [23] are a standard way to add bursts to a Poisson arrival stream. Production traces such as the Azure Functions trace show daily and weekly patterns [24]. Scaling a stateful store also has a cost of its own. Redis Cluster moves hash slots between nodes online, and keys are locked briefly while they migrate [5], [6]. Rocksteady shows that live migration in an in-memory store can keep latency low under load [25]. We use these works to choose the structure of our simulator. We found no published queueing model validated against Redis latency, so the simulator's parameters have to be measured on a Redis cluster. That calibration is pending, and every parameter value in the current simulator is provisional.

d) Quantum reinforcement learning and hybrid models.: Jerbi et al. [10] train parametrized quantum policies with few qubits on benchmark environments and prove a separa-

tion from classical learners on tasks constructed around the hardness of the discrete logarithm. The separation holds for those constructed tasks and says nothing about autoscaling. Skolik et al. [11] use VQCs as Q-functions in deep Q-learning. Their ablations show that architectural and hyperparameter choices matter more than the parameter count, which is why we compare Sofia with a classical control of the same size. Mari et al. [12] attach a variational circuit to a pre-trained classical network, a form of transfer learning, and run it on quantum hardware for image and state classification. Chen et al. [26] place VQCs inside the gates of an LSTM and report faster convergence in some of their tests. The work is a preprint, and the reported gains are specific to those test cases. Bitsakos et al. [27] ran machine learning algorithms, among them k-means, on a quantum cloud offering. None of the five quantum studies cited in this paragraph addresses autoscaling, where the effect of an action is delayed, the reward trades cost against latency, and the policy must eventually run against a real cluster within the control interval.

In summary, each of these lines of work supplies one part of our study: the threshold baseline, the DInos agent that Sofia extends, the structure of the simulator, and the variational circuit that replaces part of the Q-network.

IV. System Design

V. Simulator v2

VI. Agents and Baselines

VII. Quantum Components

VIII. Evaluation

IX. Discussion

X. Conclusion

References

[1] J. Dean and L. A. Barroso, "The tail at scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013, verified 2026-10-16 via https://api.crossref.org/works/10.1145/2408776.2408794. [Online]. Available: https://doi.org/10.1145/2408776.2408794

[2] The Kubernetes Authors, "Horizontal pod autoscaling," 2026, official documentation; accessed 2026-10-17. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

[3] Kubernetes SIG Scalability, "Kubernetes scalability SLIs/SLOs," 2026, official project document; accessed 2026-10-17. [Online]. Available: https://github.com/kubernetes/community/blob/master/sig-scalability/slos/slos.md

[4] T. Harter, B. Salmon, R. Liu, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "Slacker: Fast distribution with lazy docker containers," in *14th USENIX Conference on File and Storage Technologies (FAST 16)*, 2016, pp. 181–195, verified 2026-10-17 via https://www.usenix.org/node/194431 (abstract read); no DOI. [Online]. Available: https://www.usenix.org/node/194431

[5] Redis Ltd., "Scale with redis cluster," 2026, official documentation, not peer reviewed; accessed 2026-10-17. [Online]. Available: https://redis.io/docs/latest/operate/oss_and_stack/management/scaling/

[6] ——, "Redis cluster specification," 2026, official documentation, not peer reviewed; accessed 2026-10-17. [Online]. Available: https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/

[7] C. Bitsakos, I. Konstantinou, and N. Koziris, "DERP: A deep reinforcement learning cloud system for elastic resource provisioning," in *2018 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, 2018, pp. 21–29, verified: Crossref 10.1109/CloudCom2018.2018.00020. [Online]. Available: https://doi.org/10.1109/CloudCom2018.2018.00020

[8] C. Bitsakos, D. Tsoumakos, I. Konstantinou, and N. Koziris, "DInos: A deep reinforcement learning approach to generalizable autoscaling in stateless cloud applications," in *Database and Expert Systems Applications (DEXA 2025)*, ser. Lecture Notes in Computer Science. Springer, 2025, pp. 260–275, verified: Crossref 10.1007/978-3-032-02049-9_20. [Online]. Available: https://doi.org/10.1007/978-3-032-02049-9_20

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, verified 2026-10-16 via https://api.crossref.org/works/10.1162/neco.1997.9.8.1735. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735

[10] S. Jerbi, C. Gyurik, S. C. Marshall, H. J. Briegel, and V. Dunjko, "Parametrized quantum policies for reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS 2021)*, 2021, verified 2026-10-16 via https://arxiv.org/abs/2103.05577 (arXiv comment: NeurIPS 2021 camera-ready; proceedings pages not checked). [Online]. Available: https://arxiv.org/abs/2103.05577

[11] A. Skolik, S. Jerbi, and V. Dunjko, "Quantum agents in the gym: a variational quantum algorithm for deep q-learning," *Quantum*, vol. 6, p. 720, 2022, verified 2026-10-16 via https://arxiv.org/abs/2103.15084. [Online]. Available: https://arxiv.org/abs/2103.15084

[12] A. Mari, T. R. Bromley, J. Izaac, M. Schuld, and N. Killoran, "Transfer learning in hybrid classical-quantum neural networks," *Quantum*, vol. 4, p. 340, 2020, verified 2026-10-16 via https://arxiv.org/abs/1912.08278. [Online]. Available: https://arxiv.org/abs/1912.08278

[13] T. Lorido-Botran, J. Miguel-Alonso, and J. A. Lozano, "A review of auto-scaling techniques for elastic applications in cloud environments," *Journal of Grid Computing*, vol. 12, pp. 559–592, 2014, verified 2026-10-16 via https://api.crossref.org/works/10.1007/s10723-014-9314-7. [Online]. Available: https://doi.org/10.1007/s10723-014-9314-7

[14] Z. Zhou, C. Zhang, L. Ma, J. Gu, H. Qian, Q. Wen, L. Sun, P. Li, and Z. Tang, "AHPA: Adaptive horizontal pod autoscaling systems on Alibaba cloud container service for Kubernetes," 2023, verified 2026-10-17 via https://arxiv.org/abs/2303.03640 and ar5iv HTML (section Performance Model Training); preprint, peer-review status not established. [Online]. Available: https://arxiv.org/abs/2303.03640

[15] M. E. Taylor and P. Stone, "Transfer learning for reinforcement learning domains: A survey," *Journal of Machine Learning Research*, vol. 10, pp. 1633–1685, 2009, verified 2026-10-16 via https://jmlr.org/papers/v10/taylor09a.html. [Online]. Available: https://jmlr.org/papers/v10/taylor09a.html

[16] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI 2016)*, 2016, verified 2026-10-16 via https://arxiv.org/abs/1509.06461 (arXiv comment: AAAI 2016; pages not checked). [Online]. Available: https://arxiv.org/abs/1509.06461

[17] M. Harchol-Balter, *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press, 2013, verified 2026-10-16 via https://api.crossref.org/works/10.1017/CBO9781139226424. [Online]. Available: https://doi.org/10.1017/CBO9781139226424

[18] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961, verified 2026-10-16 via https://api.crossref.org/works/10.1287/opre.9.3.383. [Online]. Available: https://doi.org/10.1287/opre.9.3.383

[19] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble, "Tales of the tail: Hardware, OS, and application-level sources of tail latency," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2014, pp. 1–14, verified 2026-10-17 via https://api.crossref.org/works/10.1145/2670979.2670988; abstract via https://research.google/pubs/tales-of-the-tail-hardware-os-and-application-level-sources-of-tail-latency; PDF not machine-readable here. [Online]. Available: https://doi.org/10.1145/2670979.2670988

[20] J. Leverich and C. Kozyrakis, "Reconciling high server utilization and sub-millisecond quality-of-service," in *Proceedings of the Ninth European Conference on Computer Systems (EuroSys)*, 2014, pp. 1–14, verified 2026-10-17 via Crossref query.bibliographic (title match, DOI returned) and abstract at https://mast.stanford.edu/pubs/reconciling_high_server_utilization_and_sub_millisecond_quality_of_service/; full text not read. [Online]. Available: https://doi.org/10.1145/2592798.2592821

[21] B. Atikoglu, Y. Xu, E. Frachtenberg, S. Jiang, and M. Paleczny, "Workload analysis of a large-scale key-value store," in *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE Joint International*

Conference on Measurement and Modeling of Computer Systems, 2012, pp. 53–64, verified 2026-10-17 via https://api.crossref.org/works/10.1145/2254756.2254766; abstract via Semantic Scholar API; full text not read. [Online]. Available: https://doi.org/10.1145/2254756.2254766

[22] P. Bodik, A. Fox, M. J. Franklin, M. I. Jordan, and D. A. Patterson, "Characterizing, modeling, and generating workload spikes for stateful services," in *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*, 2010, pp. 241–252, verified 2026-10-17 via https://api.crossref.org/works/10.1145/1807128.1807166; abstract at https://www.microsoft.com/en-us/research/?p=160580; full text not read. [Online]. Available: https://doi.org/10.1145/1807128.1807166

[23] W. Fischer and K. Meier-Hellstern, "The Markov-modulated Poisson process (MMPP) cookbook," *Performance Evaluation*, vol. 18, no. 2, pp. 149–171, 1993, verified 2026-10-17 via https://api.crossref.org/works/10.1016/0166-5316(93)90035-S; metadata only, content not read. [Online]. Available: https://doi.org/10.1016/0166-5316(93)90035-S

[24] M. Shahrad, R. Fonseca, Í. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, 2020, verified 2026-10-17 via https://arxiv.org/abs/2003.03423 (authors, dataset note) and ar5iv HTML https://ar5iv.labs.arxiv.org/html/2003.03423 (sections 3.2, 3.3, 3.6); no DOI. [Online]. Available: https://www.usenix.org/conference/atc20/presentation/shahrad

[25] C. Kulkarni, A. Kesavan, T. Zhang, R. Ricci, and R. Stutsman, "Rocksteady: Fast migration for low-latency in-memory storage," in *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, 2017, pp. 390–405, verified 2026-10-17 via https://api.crossref.org/works/10.1145/3132747.3132784; abstract at https://www.flux.utah.edu/paper/kulkarni-sosp17. [Online]. Available: https://doi.org/10.1145/3132747.3132784

[26] S. Y.-C. Chen, S. Yoo, and Y.-L. L. Fang, "Quantum long short-term memory," 2020, verified 2026-10-16 via https://arxiv.org/abs/2009.01783 (preprint; no journal-ref listed). [Online]. Available: https://arxiv.org/abs/2009.01783

[27] C. Bitsakos, K. Nikas, and N. Koziris, "QuantuML: Machine learning algorithms and k-means on quantum cloud offering," in *Algorithmic Aspects of Cloud Computing*, ser. Lecture Notes in Computer Science. Springer, 2025, pp. 117–129, verified: Crossref 10.1007/978-3-031-94677-6_9. [Online]. Available: https://doi.org/10.1007/978-3-031-94677-6_9